

# Intents no Android

Professor: Henrique Galati  
IFSP - Campus Araraquara  
Análise e Desenvolvimento de Sistemas

No Android, uma **Intent** é um objeto que **representa uma intenção de ação fora da Activity** (como por exemplo, abrir uma outra Activity) **ou fora até mesmo do seu aplicativo** (abrir o telefone, navegador, galeria, etc.). As Intents são fundamentais para a navegação entre telas (*Activities*) e para a interação com componentes internos e externos ao app. Elas funcionam como "mensagens" que o sistema interpreta e executa conforme o contexto. Existem dois tipos de **Intent**: as **explícitas** e as **implícitas**.

---

## Tipos de Intents: Intents Explícitas

São aquelas em que **indicamos diretamente a Activity de destino**. São utilizadas quando queremos abrir uma tela específica do nosso próprio aplicativo.

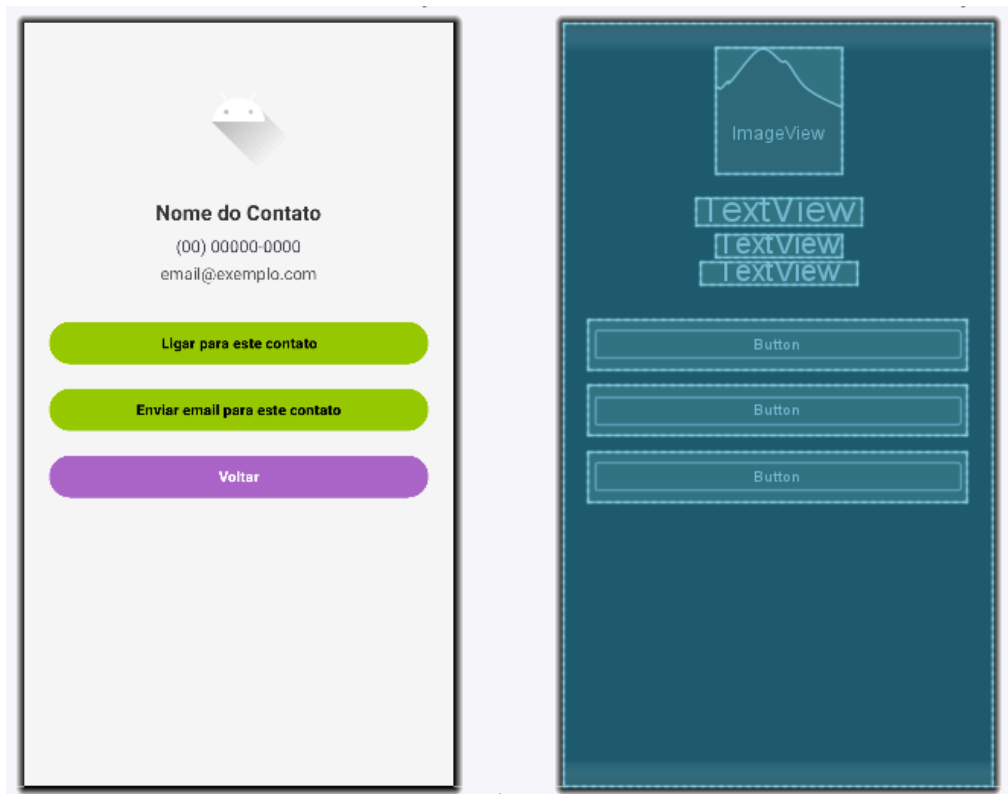
Por exemplo, imagine que você tenha criado uma nova Activity (arquivo de layout e classe), chamada **DetalheContatoActivity**. Para chamar esta nova Activity da classe **MainActivity**, basta instanciar uma Intent e inicializá-la (geralmente dentro de algum listener):

```
val intent = Intent(this, DetalheContatoActivity::class.java)
startActivity(intent)
```

---

## Exemplo prático: Criando a **DetalheContatoActivity**

Vamos criar a nossa **segunda tela** dentro do projeto. Crie o arquivo de layout **activity\_detalhe\_contato.xml** e a Activity **DetalheContatoActivity**.



## res/layout/activity\_detalhe\_contato.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#F9F9F9"
    android:padding="24dp"
    android:gravity="center_horizontal">

    <ImageView
        android:id="@+id/imgFoto"
        android:layout_width="120dp"
        android:layout_height="120dp"
        android:src="@drawable/ic_launcher_foreground"
        android:scaleType="centerCrop"
        android:layout_marginBottom="24dp" />

    <TextView
        android:id="@+id/tvNome"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Nome do Contato"
        android:textSize="20sp"
        android:textStyle="bold"
        android:textColor="#333"
        android:layout_marginBottom="8dp" />
```

```

<TextView
    android:id="@+id/tvTelefone"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="(00) 00000-0000"
    android:textSize="16sp"
    android:layout_marginBottom="4dp" />

<TextView
    android:id="@+id/tvEmail"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="email@exemplo.com"
    android:textSize="16sp"
    android:textColor="#555"
    android:layout_marginBottom="32dp" />

<Button
    android:id="@+id/btnLigar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Ligar para este contato"
    android:backgroundTint="@android:color/holo_green_light"
    android:textColor="#000"
    android:textStyle="bold"
    android:elevation="4dp"
    android:layout_marginBottom="16dp" />

<Button
    android:id="@+id/btnEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Enviar email para este contato"
    android:backgroundTint="@android:color/holo_green_light"
    android:textColor="#000"
    android:textStyle="bold"
    android:elevation="4dp"
    android:layout_marginBottom="16dp" />

<Button
    android:id="@+id/btnVoltar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Voltar"
    android:backgroundTint="@android:color/holo_purple"
    android:textColor="#FFF"
    android:textStyle="bold"
    android:elevation="2dp" />
</LinearLayout>

```

Lembrando que não estamos usando o `strings.xml`, `colors.xml` e `themes.xml` neste exemplo por não ser o foco da aula.

Vamos agora implementar a classe que representará esta nova tela.

## java/ui/DetalheContatoActivity.kt

```
class DetalheContatoActivity : AppCompatActivity() {

    private lateinit var binding: ActivityDetalheContatoBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityDetalheContatoBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setupListeners()
    }
    private fun setupListeners() {
        binding.btnLigar.setOnClickListener {
            Toast
                .makeText(this, "Ligar para o contato", Toast.LENGTH_SHORT)
                .show()
        }

        binding.btnEmail.setOnClickListener {
            Toast
                .makeText(this, "Enviar email para o contato", Toast.LENGTH_SHORT)
                .show()
        }

        binding.btnVoltar.setOnClickListener {
            Toast.makeText(this, "Voltar", Toast.LENGTH_SHORT).show()
        }
    }
}
```

Por enquanto, esta classe não faz nada além de exibir **Toasts**, além também de não poder ser chamada da **MainActivity**. Vamos refatorar a **MainActivity** para, ao clicar em algum item da **ListView**, ele chamar esta nova tela. Em **setupListeners()** da **MainActivity**:

```
fun setupListeners() {
    binding.listViewContatos.setOnItemClickListener { _, _, position, _ ->
        val intent = Intent(this, DetalheContatoActivity::class.java)
        startActivity(intent)
    }
}
```

Ao navegarmos para uma nova **Activity** usando uma **Intent**, o sistema Android automaticamente adiciona essa nova tela à **pilha de atividades**. Por isso, ao pressionar o **botão Voltar do próprio sistema**, a **Activity** atual é **finalizada** e voltamos para a anterior (no caso, a **MainActivity**).

Entretanto, podemos oferecer um **botão de Voltar próprio na interface**, como uma forma adicional de permitir que o usuário retorne.

No layout da `DetalheContatoActivity`, adicionamos um botão com o ID `btnVoltar`. Ao clicar nesse botão, usamos o método `finish()`, que encerra a Activity atual:

```
binding.btnVoltar.setOnClickListener {  
    finish()  
}
```

Por fim, no `AndroidManifest`, é necessário declarar e exportar a nova Activity, logo após a declaração `MainActivity`. Caso isso não seja feito, no momento de abrir a nova tela, seu aplicativo irá fechar.

```
<activity  
    android:name=".ui.DetalheContatoActivity"  
    android:exported="true">  
</activity>
```

Execute e teste a aplicação. Perceba que, por enquanto, esta nova Activity não faz muita coisa, inclusive, **não recebe nenhum dado**.

---

## Enviando dados para outra Activity

No Android, ao navegar de uma tela para outra (*Activity*), é comum **enviar dados juntos** — por exemplo, quando o usuário clica em um contato da lista e queremos abrir uma nova tela mostrando todos os detalhes desse contato. Para isso, usamos a classe `Intent`, que além de iniciar uma nova tela, também permite **incluir dados extras** utilizando o método `putExtra(...)`. Internamente, a classe `Intent` possui um objeto do tipo `Bundle`, capaz de armazenar estes dados extras. Um `Bundle` é uma **estrutura de dados utilizada no Android para armazenar pares de chave e valor**, como se fosse uma "mochila de informações" que você pode carregar entre Activities. É muito usado para **enviar e receber dados durante a navegação entre telas**.

### Como usar o método `putExtra(...)`?

Vamos refatorar o código para passar os dados de `MainActivity` para `DetalheContatoActivity`. Na `MainActivity`:

```
private fun setupListeners() {  
    binding.listViewContatos.setOnItemClickListener { _, _, position, _ ->  
        val contato = contatos[position]
```

```

        val intent = Intent(this, DetalheContatoActivity::class.java)
        intent.putExtra("nome", contato.nome)
        intent.putExtra("telefone", contato.telefone)
        intent.putExtra("email", contato.email)
        intent.putExtra("foto", contato.foto)
        startActivity(intent)
    }
}

```

Na `DetalheContatoActivity`, vamos implementar agora o método `setupViews()`, que será usado para receber os dados e vinculados às views desta nova tela:

```

private fun setupViews()
{
    binding.tvNome.text = intent.getStringExtra("nome")
    binding.tvTelefone.text = intent.getStringExtra("telefone")
    binding.tvEmail.text = intent.getStringExtra("email")
    binding.imgFoto.setImageResource(intent.getIntExtra("foto", R.drawable.ic_launcher_foreground))
}

```

Quando a `MainActivity` chama `startActivity(intent)`, este objeto `intent` é recebido automaticamente pela `DetalheContatoActivity`, não sendo necessário instanciarmos este objeto.

## O que pode ser enviado via `putExtra(...)`?

O método `putExtra()` aceita tipos simples de dados ou objetos que possam ser serializados. Veja abaixo os tipos mais comuns que podem ser enviados:

| Tipo de dado                      | Exemplos                                                                                               |
|-----------------------------------|--------------------------------------------------------------------------------------------------------|
| Tipos primitivos                  | <code>Int</code> , <code>Double</code> , <code>Float</code> , <code>Boolean</code> , <code>Char</code> |
| Strings                           | <code>String</code> , <code>Array&lt;String&gt;</code>                                                 |
| Objetos <code>Serializable</code> | Classes que implementam <code>java.io.Serializable</code> (veremos abaixo)                             |
| Objetos <code>Parcelable</code>   | Classes que implementam <code>Parcelable</code> (mais performático, mas não veremos aqui)              |

---

## Como transmitir um objeto de uma Activity para outra?

Quando queremos transmitir dados de uma `Activity` para outra no Android, normalmente utilizamos o método `putExtra(...)`, passando valores simples como `String`, `Int`, ou `Boolean`. No entanto, quando queremos enviar um **objeto inteiro**, como um `Contato`, precisamos garantir que esse objeto possa ser **serializado**.

### O que significa serializar?

Serializar é o processo de **converter um objeto em uma sequência de bytes** que pode ser armazenada ou transmitida. No Android, isso é útil para **enviar objetos entre telas** (Activities) usando uma `Intent`.

Para que isso seja possível, a classe do objeto deve **implementar a interface `Serializable`**, que informa ao sistema Android que aquele objeto pode ser transformado em bytes e depois reconstruído (desserializado) na outra `Activity`.

---

## Exemplo Prático: Tornando a classe `Contato` serializável

### java/model/Contato.kt

```
data class Contato(  
    val nome: String,  
    val telefone: String,  
    val email: String,  
    val foto: Int  
) : Serializable
```

Alguns cuidados ao criar uma classe serializável:

- A interface `Serializable` é simples, mas não tão eficiente quanto `Parcelable` (mais comum em apps com alto desempenho).
- Não é possível serializar objetos complexos como `Context`, `Bitmap`, `View`, `Activity`, etc.
- Toda propriedade do objeto precisa ser serializável.

Agora, vamos refatorar o método `setupListeners()` da classe `MainActivity`:

```

fun setupListeners(){
    binding.listViewContatos.setOnItemClickListener { _, _, position, _ ->
        val intent = Intent(this, DetalheContatoActivity::class.java)
        intent.putExtra("contato", contatos[position])
        startActivity(intent)
    }
}

```

Já na classe `DetalheContatoActivity`, vamos adicionar um atributo para manter o objeto contato, uma função para carregar os dados vindos da `MainActivity` (`loadData()`) e vamos refatorar o método `setupViews()`:

```

class DetalheContatoActivity : AppCompatActivity() {
    private lateinit var binding: ActivityDetalheContatoBinding
    private lateinit var contato: Contato

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityDetalheContatoBinding.inflate(layoutInflater)
        setContentView(binding.root)

        loadData()
        setupViews()
        setupListeners()
    }

    private fun loadData() {
        contato = intent.getSerializableExtra("contato") as Contato
    }

    private fun setupViews() {
        binding.tvNome.text = contato.nome
        binding.tvTelefone.text = contato.telefone
        binding.tvEmail.text = contato.email
        binding.imgFoto.setImageResource(contato.foto)
    }

    private fun setupListeners() {
        binding.btnLigar.setOnClickListener {
            Toast
                .makeText(this, "Ligar para o contato", Toast.LENGTH_SHORT)
                .show()
        }

        binding.btnEmail.setOnClickListener {
            Toast
                .makeText(this, "Enviar email para o contato", Toast.LENGTH_SHORT)
                .show()
        }

        binding.btnVoltar.setOnClickListener {
            finish()
        }
    }
}

```

---



## Tipos de Intents: Intents Implícitas

São aquelas em que **não indicamos um componente específico**, e sim uma **ação a ser realizada**, como `ACTION_VIEW`, `ACTION_SEND` ou `ACTION_DIAL`. O sistema Android verifica quais apps podem realizar essa ação e exibe uma lista de escolha para o usuário, se necessário.

### Exemplo 1: Abrir o discador do telefone com o número já preenchido

```
val intent = Intent(Intent.ACTION_DIAL)
intent.data = "tel:11999999999".toUri()
startActivity(intent)
```

Essa intent solicita que o sistema abra o app de telefone com o número já preenchido no discador. O usuário ainda precisa confirmar a chamada.

### Exemplo 2: Abrir um site no navegador padrão

```
val intent = Intent(Intent.ACTION_VIEW)
intent.data = "https://arq.ifsp.edu.br".toUri()
startActivity(intent)
```

Essa intent abre o navegador com o link indicado. Se o usuário tiver mais de um navegador, o sistema solicitará que escolha um.

### Exemplo 3: Abrir uma localização no Google Maps

```
val intent = Intent(Intent.ACTION_VIEW)
intent.data = "geo:0,0?q=Instituto+Federal+de+São+Paulo+Campus+Araraquara".toUri()
startActivity(intent)
```

A URI para este caso segue o padrão `geo:latitude,longitude?q=pesquisa`. Use `"geo:0,0?q=termo+de+busca"` quando você **não souber as coordenadas exatas**, mas quiser **buscar por nome de lugar, endereço ou ponto de referência**. Use `"geo:lat,long"` sem `"?q=..."` o quando tiver as **coordenadas GPS exatas do local**.

### Exemplo 4: Enviar e-mail com destinatário preenchido

```
val intent = Intent(Intent.ACTION_SENDTO)
intent.data = "mailto:contato@ifsp.edu.br".toUri()
startActivity(intent)
```

Aqui, o Android abre um app de e-mail com o endereço já preenchido.

## Voltando ao aplicativo AgendaApp

Esta é a versão final da classe `DetalheContatoActivity`, com a implementação dos botões Ligar e Enviar email.

```
class DetalheContatoActivity : AppCompatActivity() {

    private lateinit var binding: ActivityDetalheContatoBinding
    private lateinit var contato: Contato

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityDetalheContatoBinding.inflate(layoutInflater)
        setContentView(binding.root)

        loadData()
        setupViews()
        setupListeners()
    }

    private fun loadData() {
        contato = intent.getSerializableExtra("contato") as Contato
    }

    private fun setupViews() {
        binding.tvNome.text = contato.nome
        binding.tvTelefone.text = contato.telefone
        binding.tvEmail.text = contato.email
        binding.imgFoto.setImageResource(contato.foto)
    }

    private fun setupListeners() {
        binding.btnLigar.setOnClickListener {
            val intent = Intent(Intent.ACTION_DIAL)
            intent.data = "tel:${contato.telefone}".toUri()
            startActivity(intent)
        }

        binding.btnEmail.setOnClickListener {
            val intent = Intent(Intent.ACTION_SENDTO)
            intent.data = "mailto:${contato.email}".toUri()
            startActivity(intent)
        }

        binding.btnVoltar.setOnClickListener {
            finish()
        }
    }
}
```